Throttle in JavaScript — Detailed Explanation

Definition
Throttling limits how often a function is executed: it ensures a function runs at most once every *delay* milliseconds. When many events occur repeatedly, throttling lets the function run periodically (once per interval) instead of on every event.

Common Use Cases

- Scroll event handlers (update position or lazy-load data periodically).
- Window resize (recompute layout at intervals, not continuously).
- Mousemove or pointer events (sample position every X ms).
- Prevent button spam / rate-limit actions (allow action once per interval).
- Polling/heartbeat scheduling where a function should not run faster than a rate.

Timestamp-based Throttle (Code)

```
// Throttle implementation (timestamp-based)
function throttle(func, delay) {
  let lastCall = 0; // timestamp (ms) when func last ran

  return function (...args) {
    const now = Date.now();            // current timestamp in ms
    if (now - lastCall >= delay) {    // enough time has passed since last run
      lastCall = now;                 // update last run time
      func.apply(this, args);         // call original function
    }
  };
}
```

Step-by-step: timestamp-based throttle (concrete epoch example)

Parameters: - delay = 2000 ms - initial lastCall = 0 (meaning "no previous run recorded") We simulate three incoming calls at these exact epoch timestamps (ms since Unix epoch): 1) Call 1 time: - epoch ms: 1756613210075 - human time: 2025-08-31 04:06:50.075 UTC - Calculation: now - lastCall = 1756613210075 - 0 = 1756613210075 - Compare to delay: 1756613210075 >= 2000 → TRUE (because epoch is a very large number) - Action: Function runs. Set lastCall = 1756613210075. Output: RUN at 1756613210075 (2025-08-31 04:06:50.075 UTC) 2) Call 2 time (1,000 ms after Call 1): - epoch ms: 1756613211075 - human time: 2025-08-31 04:06:51.075 UTC - Calculation: now - lastCall = 1756613211075 - 1756613210075 = 1000 ms - Compare to delay: 1000 >= 2000 → FALSE - Action: Since condition is False, function is NOT called (THROTTLED / IGNORED). lastCall stays = 1756613210075. 3) Call 3 time (3,200 ms after Call 1; 2,200 ms after Call 2): - epoch ms: 1756613213275 - human time: 2025-08-31 04:06:53.275 UTC - Calculation: now - lastCall = 1756613213275 - 1756613210075 = 3200 ms - Compare to delay: 3200 >= 2000 → TRUE - Action: Condition True → function runs. Set lastCall = 1756613213275. Output: RUN at 1756613213275 (2025-08-31 04:06:53.275 UTC) Summary of this run: - Function executed at Call 1 (epoch 1756613210075) and Call 3 (epoch 1756613213275). - Call 2 was ignored because only 1000 ms elapsed since the last execution (less than the 2000 ms delay).

```
Timeline (delay = 2000 ms):
Call times (epoch ms) -> human (UTC)
1756613210075 -> 2025-08-31 04:06:50.075 UTC   (RUN)
1756613211075 -> 2025-08-31 04:06:51.075 UTC   (IGNORED: only 1000 ms since last run)
1756613213275 -> 2025-08-31 04:06:53.275 UTC   (RUN: 3200 ms since last run >= 2000)

Notes:
- Because lastCall starts at 0, the very first real-world call will almost always run immediately.
- The throttle enforces a minimum interval of 2000 ms between allowed executions.
```

Timer-based throttle (alternative implementation)

```
// Timer-based throttle: runs immediately, then blocks via a timer
function throttle(func, delay) {
  let timer = null;
  return function (...args) {
    if (!timer) {
      func.apply(this, args); // immediate run
      timer = setTimeout(() => { timer = null; }, delay); // block further runs until timer clears
    }
  };
}
```

Leading + Trailing (hybrid) and helpers

- You can combine timestamp and timer approaches to support both a leading call (run immediately) and a trailing call (run once more after the burst) — useful for UI responsiveness. - Common helpers: `.cancel()` to drop scheduled trailing call, `.flush()` to run it immediately. - Lodash's _.throttle provides robust options for leading/trailing behavior — consider using it in production.

Gotchas & Tips

- Use a normal function for the wrapper (not an arrow) if you need to preserve `this`. - Keep a reference to the throttled function to remove event listeners later. - For high-precision timing (sub-ms), consider performance.now() instead of Date.now(). - Validate the delay parameter; delays <= 0 will effectively disable throttling.

End of document — Throttle overview with concrete epoch example